React — Multi-Page App with React Router

Comprehensive Lesson Guide & Practical Reference

1. The Evolution: Traditional Web vs. React Router

In traditional multi-page web applications, navigating to a new URL requires requesting a completely new HTML page from the server. React Router transforms this experience into a Single Page Application (SPA).

Traditional Way (Railway Ticket Counter)	React Router Way (Express Mall Food Court)
Slow & Sequential: Every page transition requires waiting in line to fetch a full HTML page from the server.	**Instant Navigation:** Like walking directly between food stalls inside a mall, navigation happens instantly without leaving the app.
Full Page Reload: The browser completely refreshes, causing a disruption in state and user experience.	**Dynamic Render:** Only the necessary components update on screen; no page refresh occurs.

2. React Router's Four Magic Components

React Router relies on four core elements to manage seamless routing throughout an application:

1. BrowserRouter

The Mall Building

Wraps the entire application. It tracks browser history and keeps the UI synced with the current URL bar.

2. Routes

The Metro Line Map

Acts as the container for all individual route definitions. It analyzes current URL paths to render the matching child route.

3. Route

Each Metro Stop

Defines a single mapping between a specific

4. Link

The Magic Elevator

Replaces standard `<a>` tags. It allows users

URL path (e.g., `/menu`) and the component to render on screen.

to navigate around the app without triggering full page reloads.

3. Case Study: Sharmaji's Digital Sweet Shop

We build a multi-page web app for "Sharmaji's Digital Sweet Shop" containing 3 primary pages:

- **Home (`/`):** Welcome page introducing the sweet shop.
- **Menu (`/menu`):** Catalog displaying all available sweets and pricing.
- **Contact (`/contact`):** Location map and contact details for Sharmaji.

Code Implementation Guide

4. Step-by-Step Implementation

Step A: Setting up `BrowserRouter`

Wrap your top-level component inside `<BrowserRouter>` to enable client-side routing across the application.

```
import { BrowserRouter } from 'react-router-dom';
import Navbar from './Navbar';
import MainContent from './MainContent';

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <MainContent />
    </BrowserRouter>
  );
}

export default App;
```

Step B: Defining Routes and Mapping Components

Use `<Routes>` and `<Route>` to establish paths and assign corresponding page components.

```
import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import Menu from './pages/Menu';
import Contact from './pages/Contact';
import NotFound from './pages/NotFound';

function MainContent() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/menu" element={<Menu />} />
      <Route path="/contact" element={<Contact />} />

      {/* Catch-all route for non-existent pages */}
      <Route path="*" element={<NotFound />} />
    </Routes>
```

```
  );
}
```

Step C: Navigation with `Link` Components

Always use `<Link to="...">` instead of traditional HTML anchor tags (`<a href="...">`).

```
import { Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav className="navbar">
      <Link to="/">Home</Link>
      <Link to="/menu">Menu</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
}
```

The "Auto-Driver" Fallback Route (`path="*"`):
If a user types an invalid URL (e.g., `/unknown`), the catch-all wildcard route matches the path and renders a friendly "404 Page Not Found" component, preventing broken states or blank pages!

Student Practice & Review

Test your understanding of React Router core concepts

Practice Questions

Answer the following questions based on the lesson materials above. Write your answers clearly in your notebook.

Question 1 (Multiple Choice)

What is the primary role of the `BrowserRouter` component in React Router?

(A) It renders individual HTML headings and page images. (B) It wraps the application to enable client-side routing and sync with the URL bar. (C) It executes API calls to backend databases automatically. (D) It styles page navigation buttons using CSS.

Question 2 (Multiple Choice)

Why should you use the `<Link>` component instead of a standard `<a>` tag for internal navigation?

(A) `<Link>` prevents full page reloads and enables instant UI transitions. (B) Standard `<a>` tags do not support text labels. (C) `<Link>` automatically downloads external assets. (D) `<a>` tags cannot be used inside React components.

Question 3 (Multiple Choice)

What path configuration is used to handle invalid/unmatched URLs (404 Not Found pages)?

(A) `<Route path="/error" element={<NotFound />} />` (B) `<Route path="*" element={<NotFound />} />` (C) `<Route path="/404" element={<NotFound />} />` (D) `<Route path="default" element={<NotFound />} />`

Question 4 (Short Answer)

Explain the analogy between a metro rail network and React Router's `Routes` and `Route` components.

Question 5 (Code Exercise)

Write a React JSX navigation bar component called `HeaderNav` that renders links to **Home** (`/`), **About** (`/about`), and **Services** (`/services`) using React Router.